MATPLOTLIB

Plotting - matplotlib

- User friendly, but powerful, plotting capabilites for python
- http://matplotlib.sourceforge.net/



- Once installed (default at Conda)

```
>>> import matplotlib.pyplot as plt
```

- Settings can be customised by editing ~/.matplotlib/matplotlibrc
  - default font, colours, layout, etc.
- Helpful website
  - many examples

Pyplot and pylab

Pylab is a module in matplotlib that gets installed alongside matplotlib; and matplotlib.pyplot is a module in matplotlib.

- **Pyplot** provides the state-machine interface to the underlying plotting library in matplotlib. This means that figures and axes are implicitly and automatically created to achieve the desired plot. Setting a title will then automatically set that title to the current axes object.
- **Pylab** combines the pyplot functionality (for plotting) with the numpy functionality (for mathematics and for working with arrays) in a single namespace, For example, one can call the sin and cos functions just like you could in MATLAB, as well as having all the features of pyplot.
- The pyplot interface is generally preferred for non-interactive plotting (i.e., scripting). The pylab interface is convenient for interactive calculations and plotting, as it minimizes typing. Note that this is what you get if you use the ipython shell with the --pylab option, which imports everything from pylab and makes plotting fully interactive.

Pylab and Pyplot

```
$ python
Python 2.7.3 (default, Aug  9 2012, 17:23:57)
[GCC 4.7.1 20120720 (Red Hat 4.7.1-5)] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> import matplotlib.pyplot as plt
>>> plt.plot([1,2,3,4])
[<matplotlib.lines.Line2D object at 0x1fe53d0>]
>>> plt.ylabel('some numbers')
<matplotlib.text.Texy object at 0x1d6ad90>
>>> plt.show()
```

```
$ ipython --pylab
Python 2.7.3 (default, Aug  9 2012, 17:23:57)
Type "copyright", "credits" or "license" for more information.

IPython 0.13.2 -- An enhanced Interactive Python.
?         -> Introduction and overview of IPython's features.
%quickref -> Quick reference.
help      -> Python's own help system.
object?   -> Details about 'object', use 'object??' for extra details.

Welcome to pylab, a matplotlib-based Python environment [backen: GTKAgg].
For more information, type 'help(pylab)'

In [1]: plot([1,2,3,4])
```

Matplotlib.pyplot example

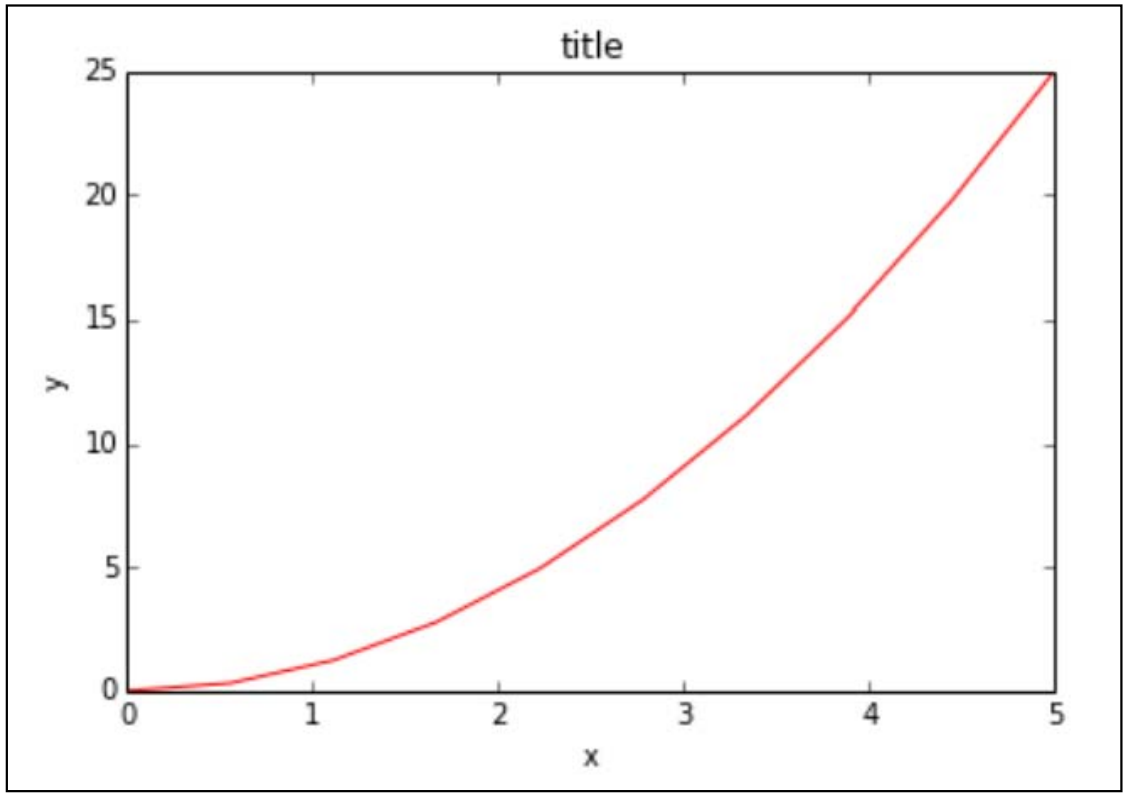
Function	Description	Function	Description	Function	Description
acorr	plot the autocorrelation function	gca	return the current axes	gca	return the current axes
annotate	annotate something in the figure	gcf	return the current figure	gcf	return the current figure
arrow	add an arrow to the axes	gci	get the current image, or None	gci	get the current image, or None
axes	create a new axes	getp	get a graphics property	getp	get a graphics property
axhline	draw a horizontal line across axes	grid	set whether gridding is on	grid	set whether gridding is on
axvline	draw a vertical line across axes	hexbin	make a 2D hexagonal binning plot	hexbin	make a 2D hexagonal binning plot
axhspan	draw a horizontal bar across axes	hist	make a histogram	hist	make a histogram
axvspan	draw a vertical bar across axes	hold	set the axes hold state	hold	set the axes hold state
axis	set or return the current axis limits	ioff	turn interaction mode off	ioff	turn interaction mode off
barbs	a (wind) barb plot	ion	turn interaction mode on	ion	turn interaction mode on
bar	make a bar chart	isinteractive	return True if interaction mode is on	isinteractive	return True if interaction mode is on
barh	a horizontal bar chart	imread	load image file into array	imread	load image file into array
broken_barh	a set of horizontal bars with gaps	imsave	save array as an image file	imsave	save array as an image file
box	set the axes frame on/off state	imshow	plot image data	imshow	plot image data
boxplot	make a box and whisker plot	ishold	return the hold state of the current ax	ishold	return the hold state of the current axes
cla	clear current axes	legend	make an axes legend	legend	make an axes legend
clabel	label a contour plot	locator_params	adjust parameters used in locating ax ticks	locator_params	adjust parameters used in locating axis ticks
clf	clear a figure window				
clim	adjust the color limits of the current imag	loglog	a log log plot	loglog	a log log plot
close	close a figure window	matshow	display a matrix in a new figure prese aspect	matshow	display a matrix in a new figure preserving aspect
colorbar	add a colorbar to the current figure				
cohere	make a plot of coherence	margins	set margins used in autoscaling	margins	set margins used in autoscaling
contour	make a contour plot	pcolor	make a pseudocolor plot	pcolor	make a pseudocolor plot
contourf	make a filled contour plot	pcolormesh	make a pseudocolor plot using a quadrilateral mesh	pcolormesh	make a pseudocolor plot using a quadrilateral mesh
csd	make a plot of cross spectral density				
delaxes	delete an axes from the current figure	pie	make a pie chart	pie	make a pie chart
draw	Force a redraw of the current figure	plot	make a line plot	plot	make a line plot
errorbar	make an errorbar graph	plot_date	plot dates	plot_date	plot dates
figlegend	make legend on the figure rather than the axes	plotfile	plot column data from an ASCII tab/space/comma delimited file	plotfile	plot column data from an ASCII tab/space/comma delimited file
figimage	make a figure image	pie	pie charts	pie	pie charts
figtext	add text in figure coords	polar	make a polar plot on a PolarAxes	polar	make a polar plot on a PolarAxes
figure	create or change active figure	psd	make a plot of power spectral density	psd	make a plot of power spectral density
fill	make filled polygons	quiver	make a direction field (arrows) plot	quiver	make a direction field (arrows) plot
fill_between	make filled polygons between two curves	rc	control the default params	rc	control the default params

Matplotlib.pyplot basic example

The easiest way to get started with plotting using matplotlib is often to use the MATLAB-like API provided by matplotlib.

```
>>> import matplotlib.pylab as plt
>>> x = plt.linspace(0, 5, 10)
>>> y = x ** 2

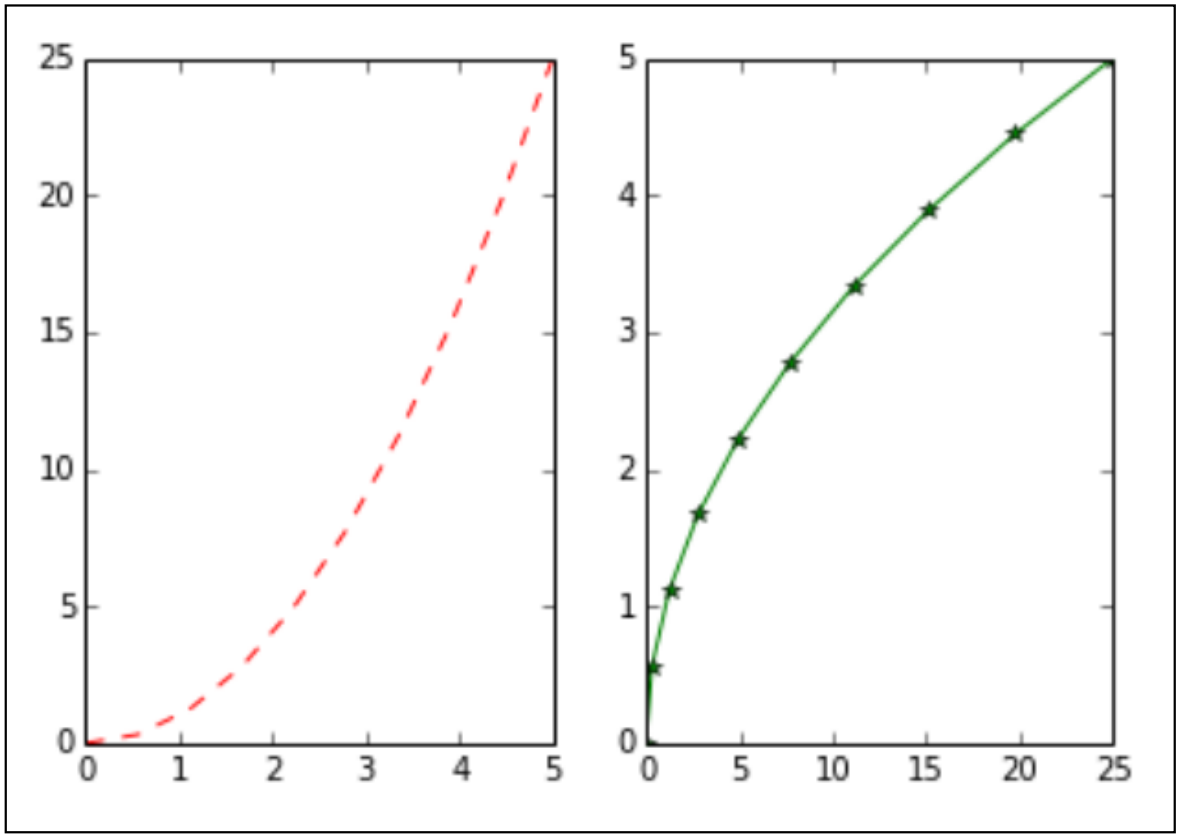
>>> figure()
>>> plot(x, y, 'r')
>>> xlabel('x')
>>> ylabel('y')
>>> title('title')
>>> show()
```



Matplotlib.pyplot basic example

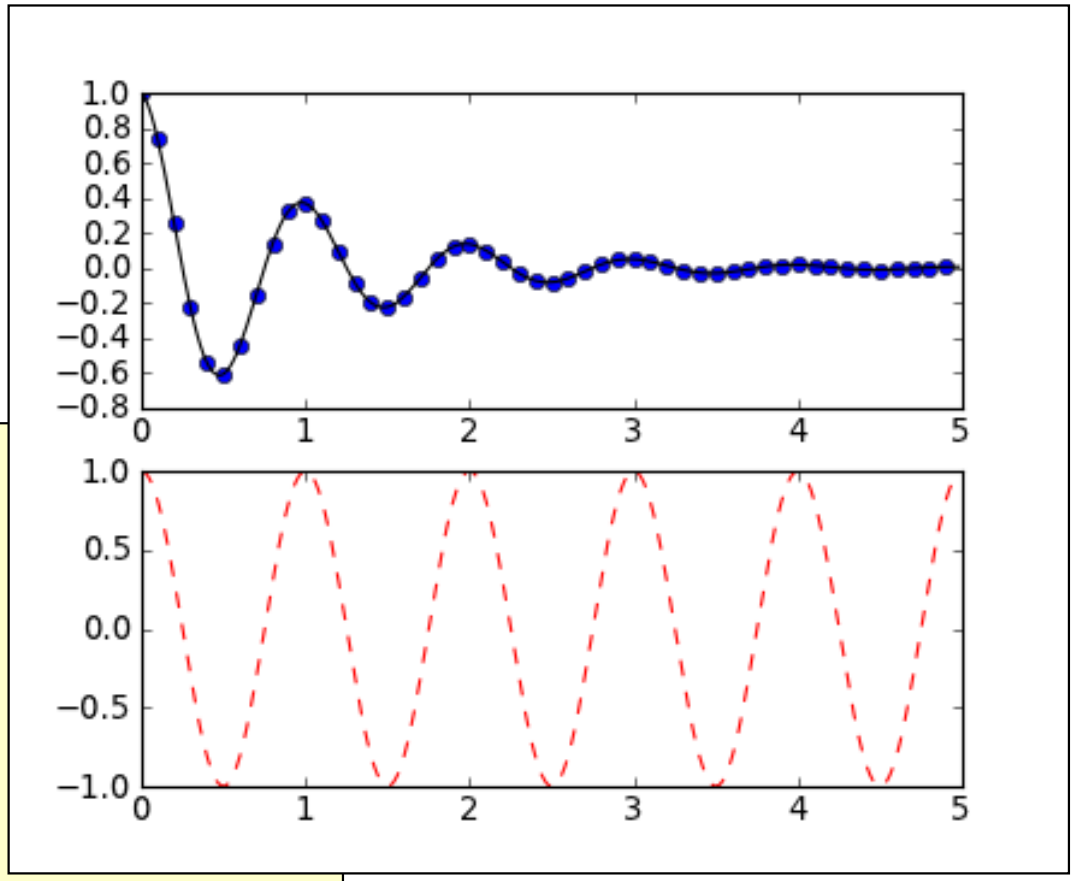
Most of the plotting related functions in MATLAB are covered by the pylab module. For example, subplot and color/symbol selection

```
>>> plt.subplot(1,2,1)
>>> plt.plot(x, y, 'r--')
>>> plt.subplot(1,2,2)
>>> plt.plot(y, x, 'g*-');
```



Matplotlib.pyplot example

Working with multiple figures and axes. The subplot() command specifies numrows, numcols, fignum, where fignum ranges from 1 to numrows*numcols.




```
import numpy as np
import matplotlib.pyplot as plt

def f(t):
    return np.exp(-t) * np.cos(2*np.pi*t)

t1 = np.arange(0.0, 5.0, 0.1)
t2 = np.arange(0.0, 5.0, 0.02)

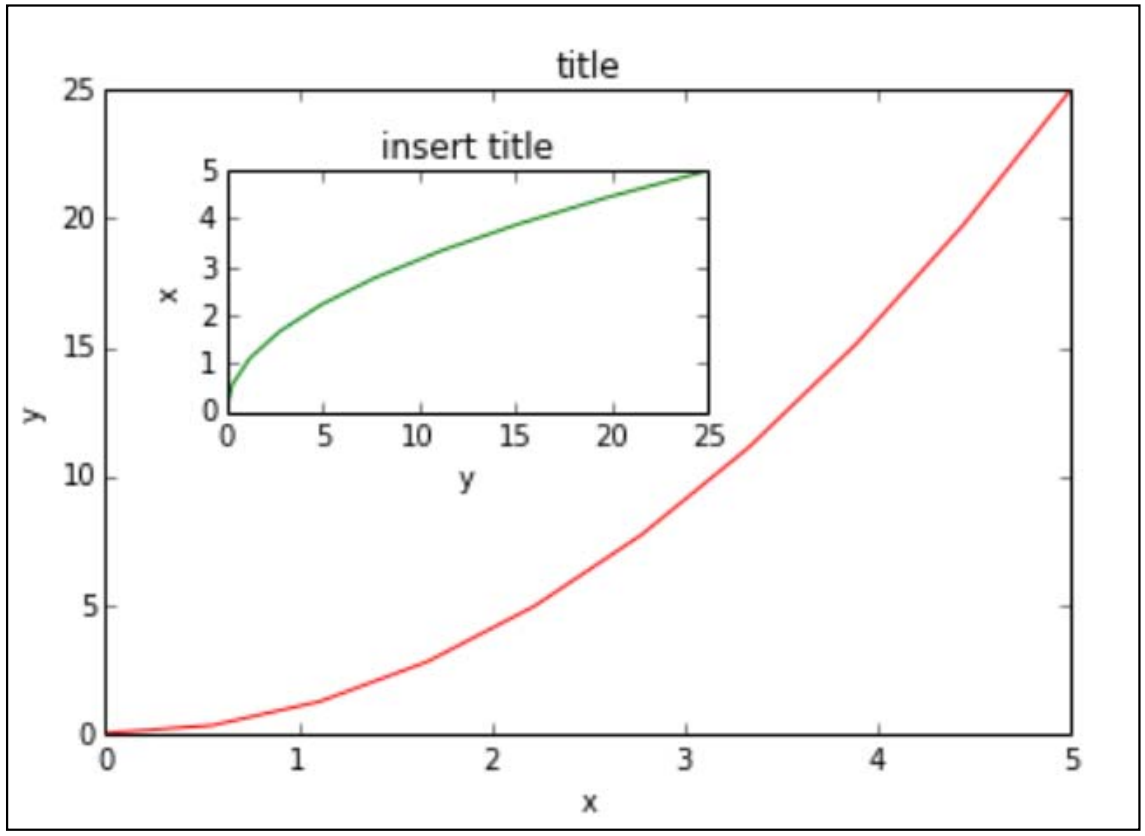
plt.figure(1)
plt.subplot(211)
plt.plot(t1, f(t1), 'bo', t2, f(t2), 'k')

plt.subplot(212)
plt.plot(t2, np.cos(2*np.pi*t2), 'r--')
```

Matplotlib.pyplot basic example

Matplotlib can be used in object oriented approach which is particularly useful when we deal with subplots

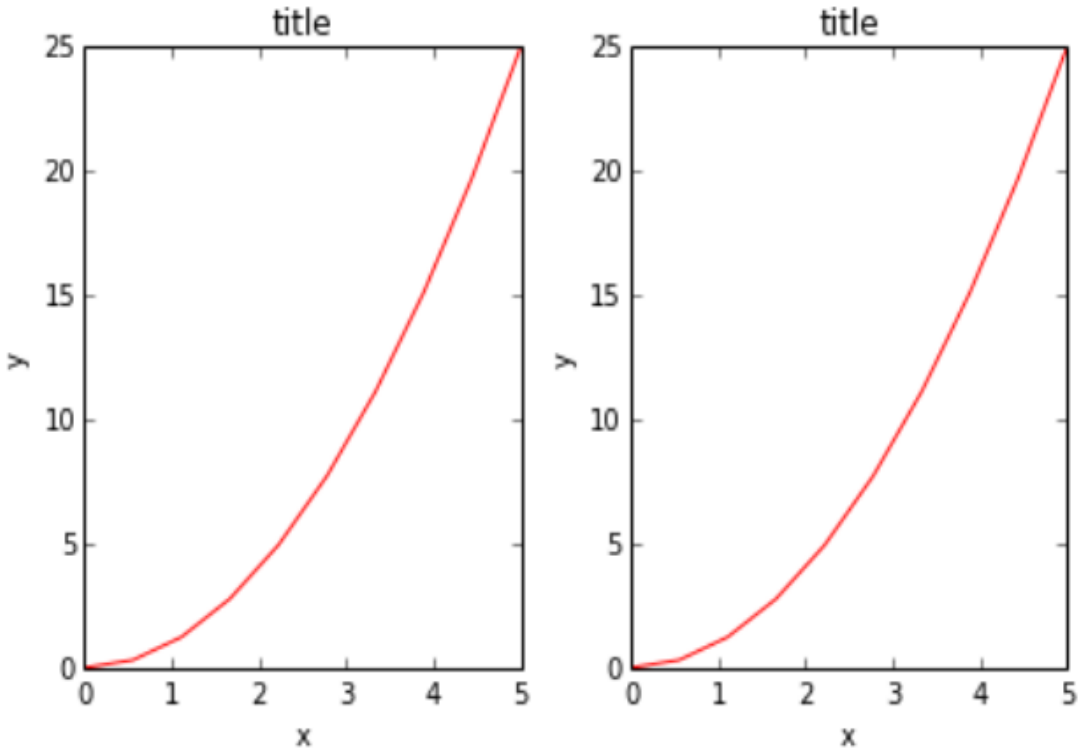
```
fig = plt.figure()
axes1 = fig.add_axes([0.1, 0.1,
0.8, 0.8]) # main axes
axes2 = fig.add_axes([0.2, 0.5,
0.4, 0.3]) # inset axes
# main figure
axes1.plot(x, y, 'r')
axes1.set_xlabel('x')
axes1.set_ylabel('y')
axes1.set_title('title')
# insert
axes2.plot(y, x, 'g')
axes2.set_xlabel('y')
axes2.set_ylabel('x')
axes2.set_title('insert title');
```



Matplotlib.pyplot basic example

With multiple plots on one screen sometimes the labels are getting in the way. Solve this with tight_layout

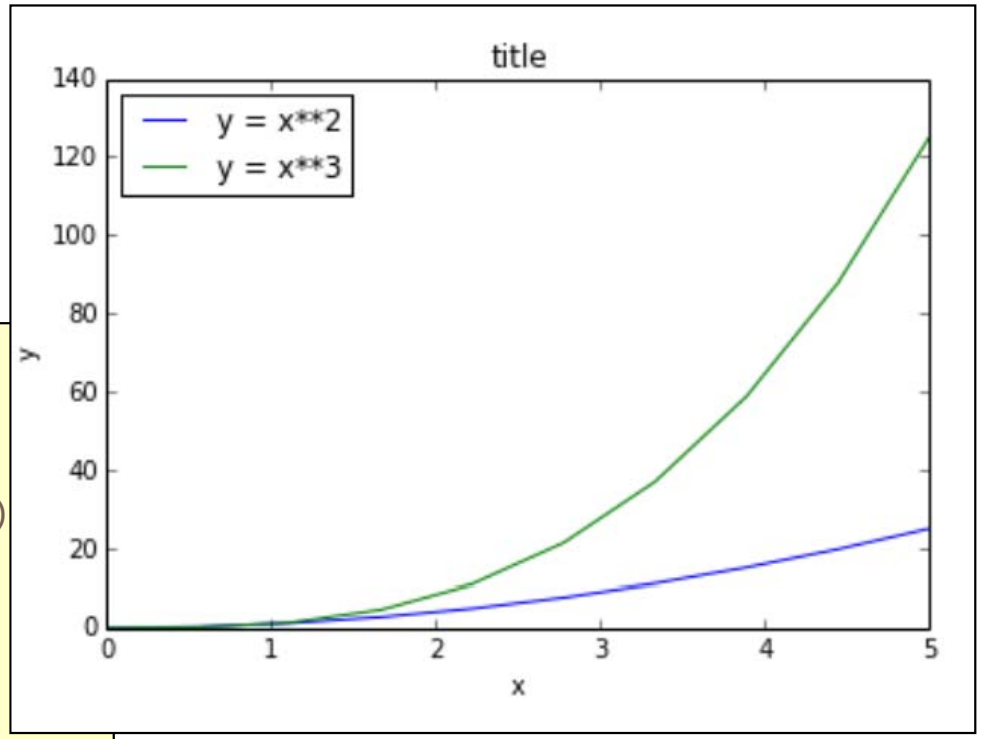
```
fig, axes = plt.subplots(nrows=1,
   ncols=2)
for ax in axes:
ax.plot(x, y, 'r')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('title')
fig.tight_layout()
```



Matplotlib.pyplot basic example

Labels and legends and titles

```
>>> fig, ax = plt.subplots()
>>> ax.plot(x, x**2, label="y = x**2")
>>> ax.plot(x, x**3, label="y = x**3")
>>> ax.legend(loc=2); # upper left corner
>>> ax.set_xlabel('x')
>>> ax.set_ylabel('y')
>>> ax.set_title('title');
```

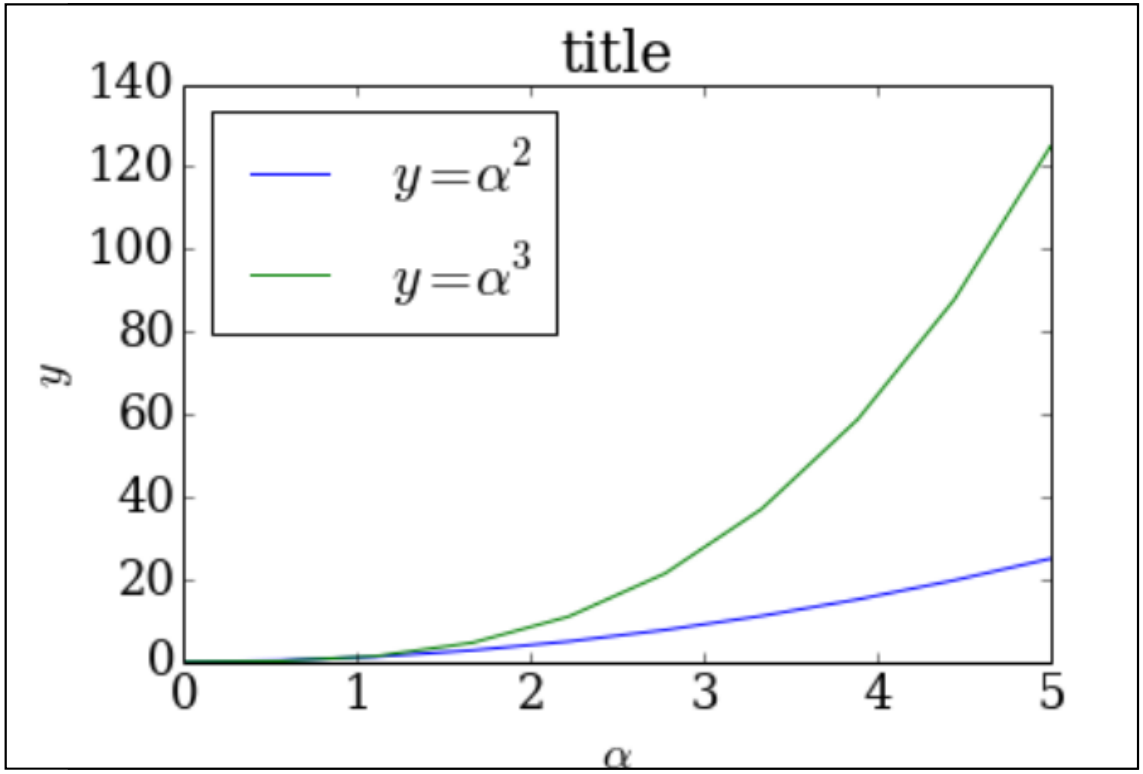



```
>>> ax.plot(x, x**2, label="curve1")
>>> ax.plot(x, x**3, label="curve2")
>>> ax.legend()

>>> ax.legend(loc=0) # let matplotlib decide
>>> ax.legend(loc=1) # upper right corner
>>> ax.legend(loc=2) # upper left corner
>>> ax.legend(loc=3) # lower left corner
>>> ax.legend(loc=4) # lower right corner
```

Matplotlib.pyplot basic example

Changing the font size and family and using LaTeX formatted tekst.



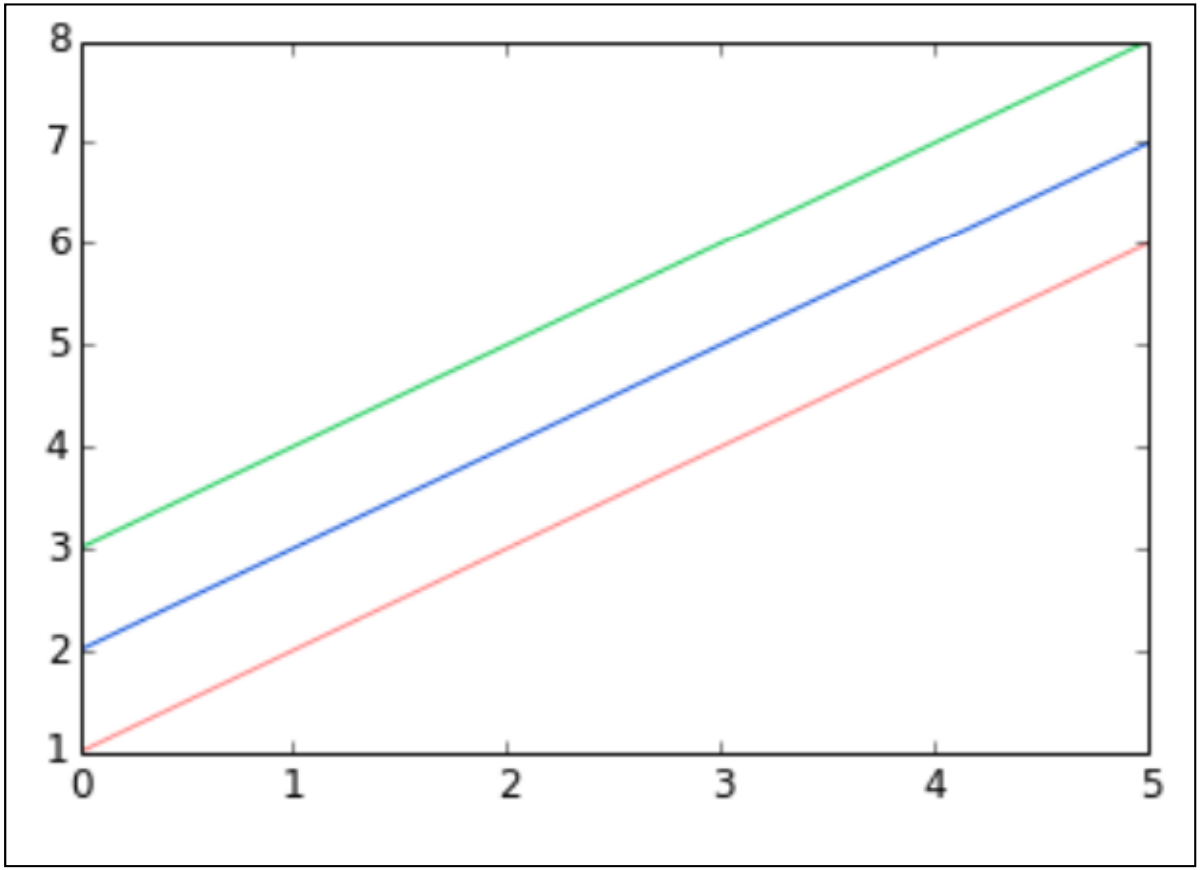

```
matplotlib.rcParams.update({'font.size': 18, 'font.family': 'serif'})
```

```
>>> fig, ax = plt.subplots()
>>> ax.plot(x, x**2, label=r"$y = \alpha^2$")
>>> ax.plot(x, x**3, label=r"$y = \alpha^3$")
>>> ax.legend(loc=2) # upper left corner
>>> ax.set_xlabel(r'$\alpha$', fontsize=18)
>>> ax.set_ylabel(r'$y$', fontsize=18)
>>> ax.set_title('title');
```

Matplotlib.pyplot basic example

You have full control over colors, linewidths and linetypes.

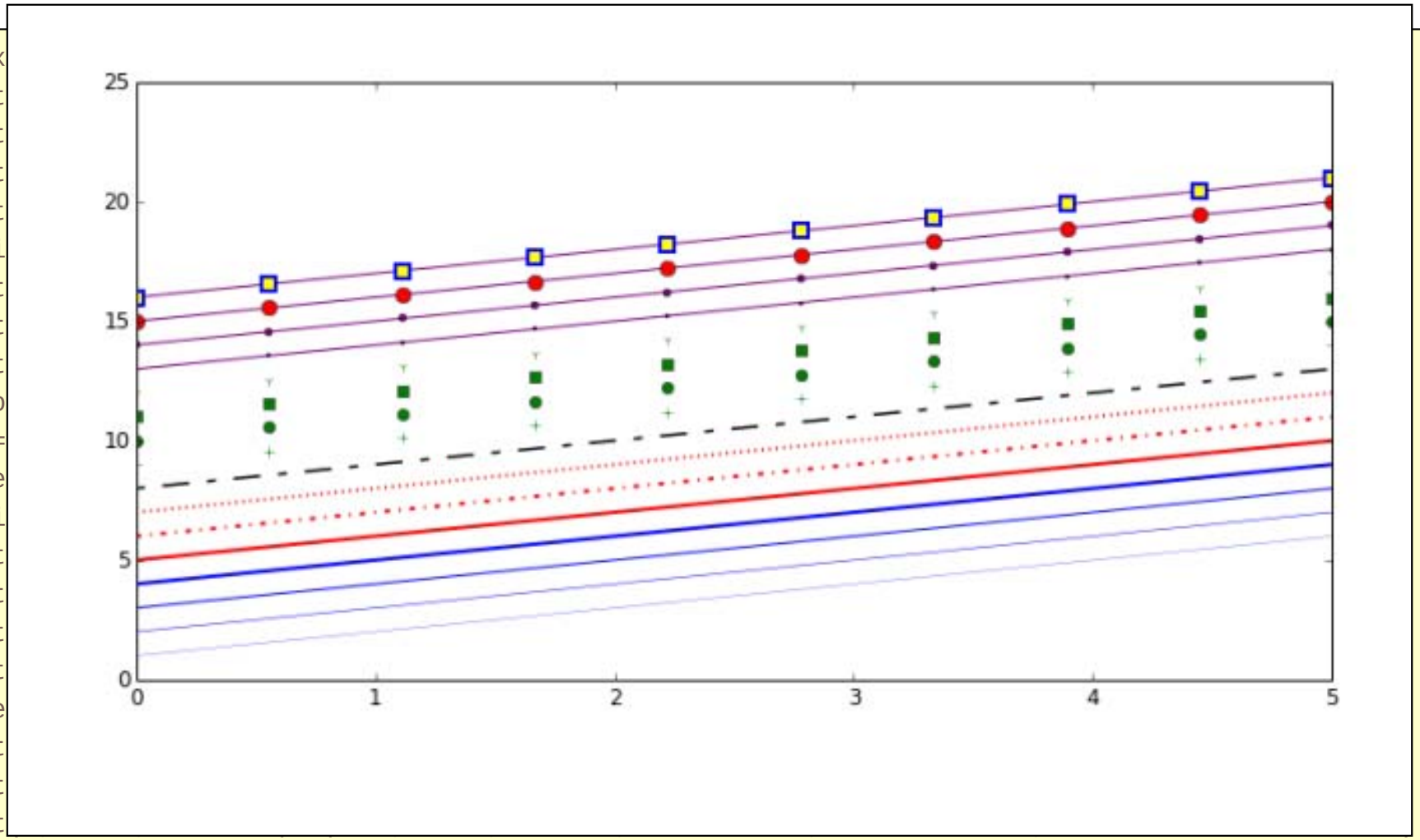
For colors one can use simple syntax like ‘b’ for blue, ‘g’ for green, etc. or RGB hex codes with transparency alpha code:




```
fig, ax = plt.subplots()
ax.plot(x, x+1, color="red", alpha=0.5) # half-transparant red
ax.plot(x, x+2, color="#1155dd") # RGB hex code for a bluish color
ax.plot(x, x+3, color="#15cc55") # RGB hex code for a greenish color
```

Matplotlib.pyplot basic example

Line and marker styles are coded:



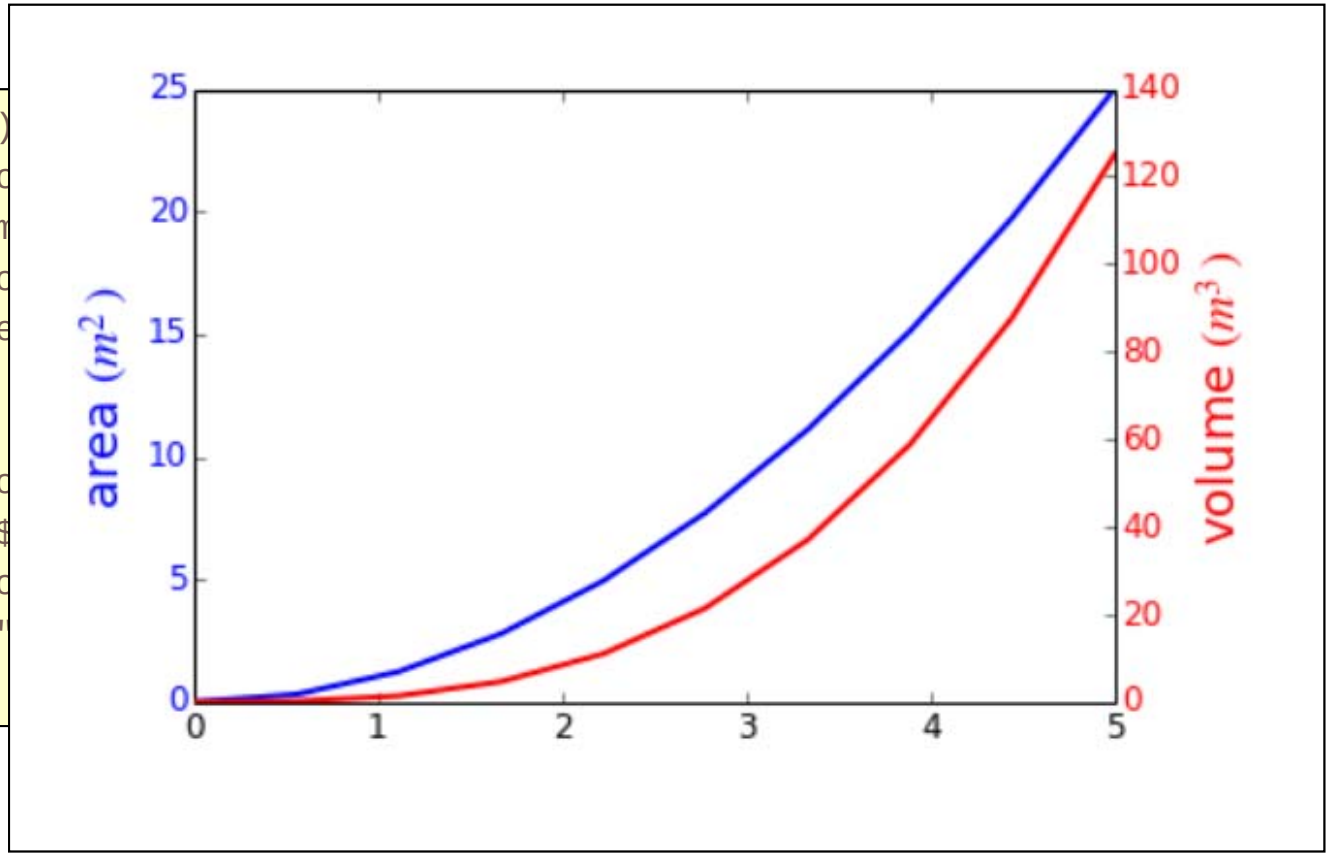

```
fig, ax
ax.plot
ax.plot
ax.plot
ax.plot
# possi
ax.plot
ax.plot
ax.plot
# custo
line, =
line.se
# possi
ax.plot
ax.plot
ax.plot
ax.plot
# marke
ax.plot
ax.plot
ax.plot
ax.plot(x, x+16, color="purple", lw=1, ls='-', marker='s', markersize=8,
markerfacecolor="yellow", markeredgewidth=2, markeredgecolor="blue");
```

Matplotlib.pyplot basic example

Sometimes it is useful to have dual x or y axes in a figure; for example, when plotting curves with different

units together. Matplotlib supports this with the twinx and twiny functions:

```
>>> fig, ax1 = plt.subplots()
>>> ax1.plot(x, x**2, lw=2, c
>>> ax1.set_ylabel(r"area $(m
>>> for label in ax1.get_ytic
...     label.set_color("blue
...
>>> ax2 = ax1.twinx()
>>> ax2.plot(x, x**3, lw=2, c
>>> ax2.set_ylabel(r"volume $
>>> for label in ax2.get_ytic
...     label.set_color("red"
```



Overwhelming annotation

```
from matplotlib.pyplot import figure, show
from matplotlib.patches import Ellipse
import numpy as np

if 1:
    fig = figure(1,figsize=(8,5))
    ax = fig.add_subplot(111, autoscale_on

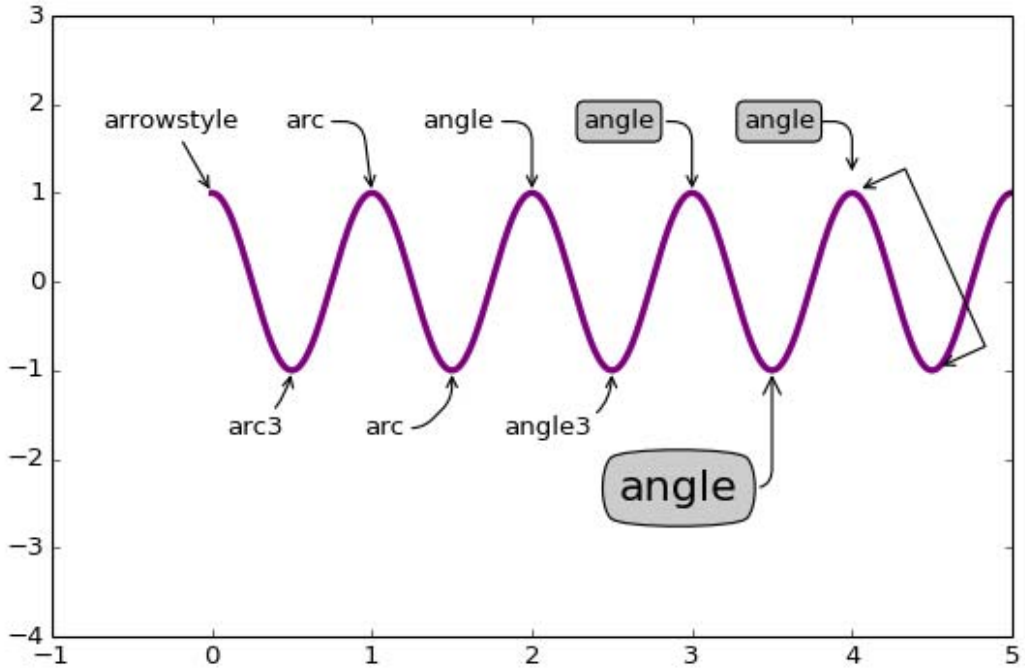
    t = np.arange(0.0, 5.0, 0.01)
    s = np.cos(2*np.pi*t)
    line, = ax.plot(t, s, lw=3, color='pur

    ax.annotate('arrowstyle', xy=(0, 1),
                xytext=(-50, 30), textcoor
                arrowprops=dict(arrowstyle
                )

    ax.annotate('arc3', xy=(0.5, -1),  xyc
                xytext=(-30, -30), textcoo
                arrowprops=dict(arrowstyle="->",
                                connectionstyle="arc3,rad=.2")
                )

    ax.annotate('arc', xy=(1., 1),  xycoords='data',
                xytext=(-40, 30), textcoords='offset points',
                arrowprops=dict(arrowstyle="->",
                                connectionstyle="arc,angleA=0,armA=30,rad=10"),
                )

    ax.annotate('arc', xy=(1.5, -1),  xycoords='data',
                xytext=(-40, -30), textcoords='offset points',
                arrowprops=dict(arrowstyle="->",
                                connectionstyle="arc,angleA=0,armA=20,angleB=-
90,armB=15,rad=7"),
```



Matplotlib.pyplot basic example

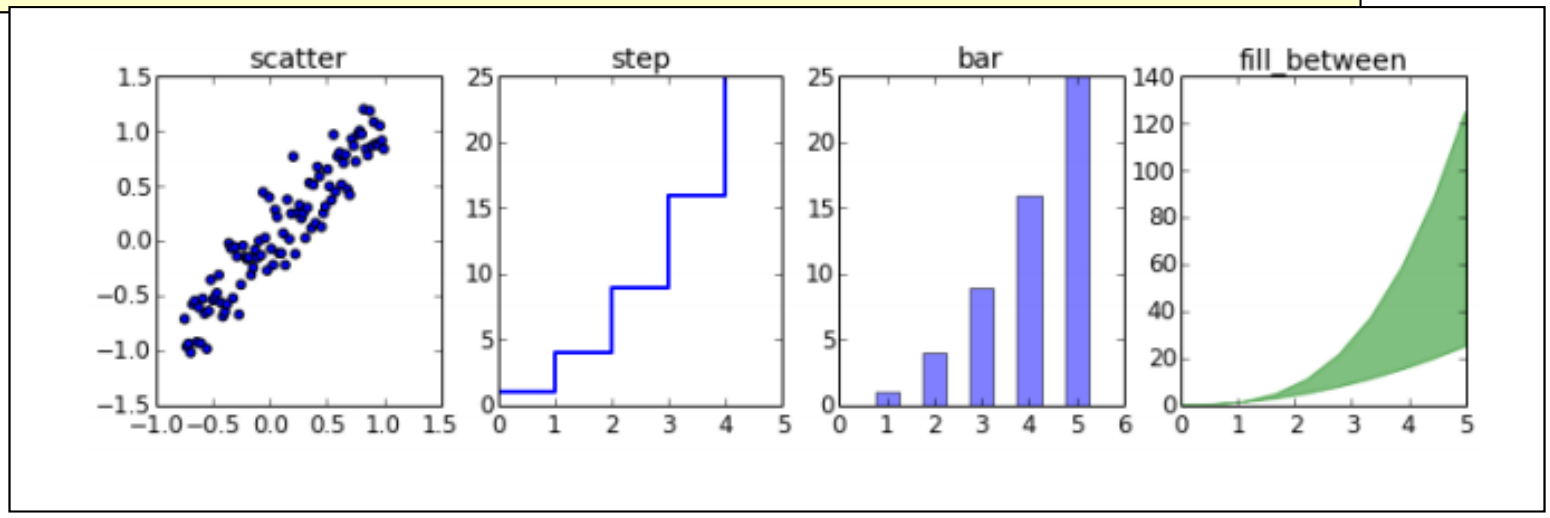
Other 2D plt styles

```
>>> n = array([0,1,2,3,4,5])
>>> fig, axes = plt.subplots(1, 4, figsize=(12,3))
>>> axes[0].scatter(xx, xx + 0.25*randn(len(xx)))
>>> axes[0].set_title("scatter")

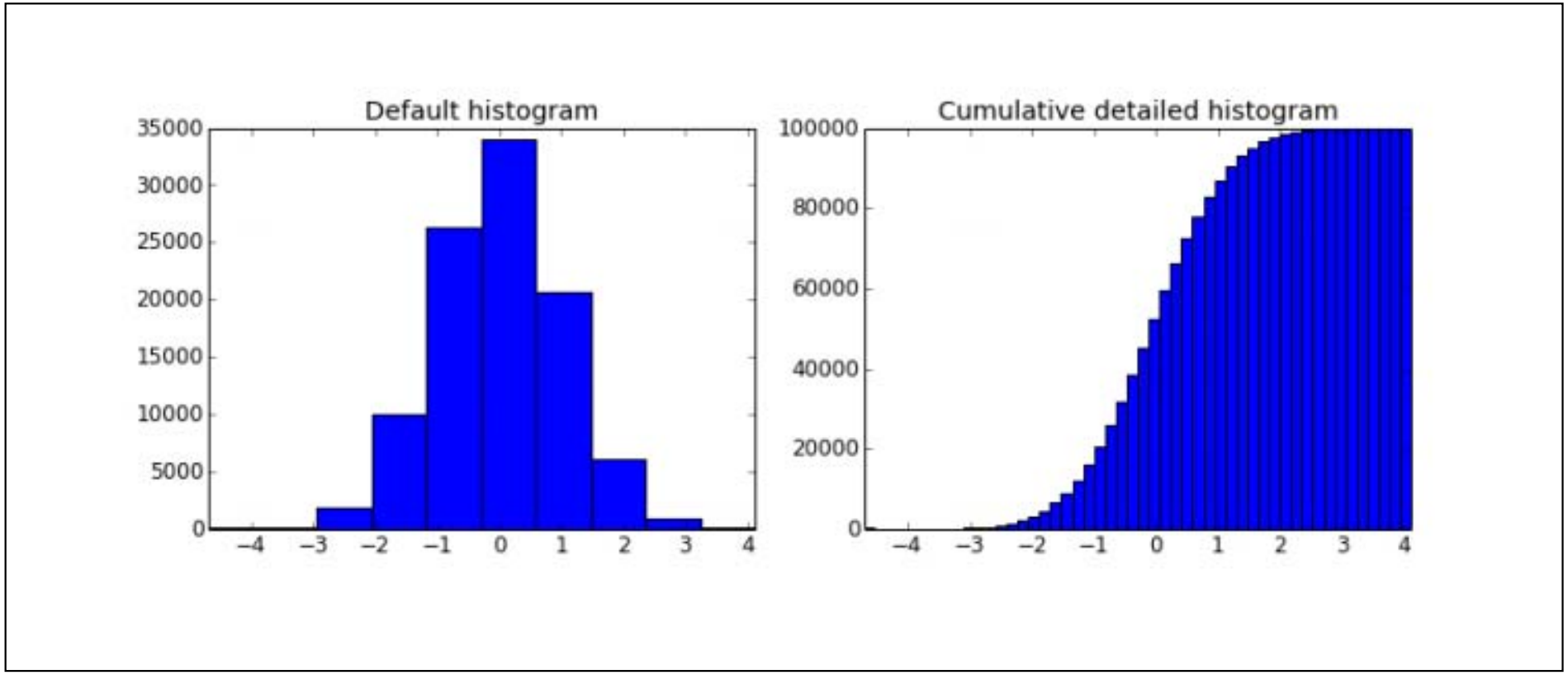
>>> axes[1].step(n, n**2, lw=2)
>>> axes[1].set_title("step")

>>> axes[2].bar(n, n**2, align="center", width=0.5, alpha=0.5)
>>> axes[2].set_title("bar")

>>> axes[3].fill_between(x, x**2, x**3, color="green", alpha=0.5)
>>> axes[3].set_title("fill_between")
```



Matplotlib.pyplot basic example



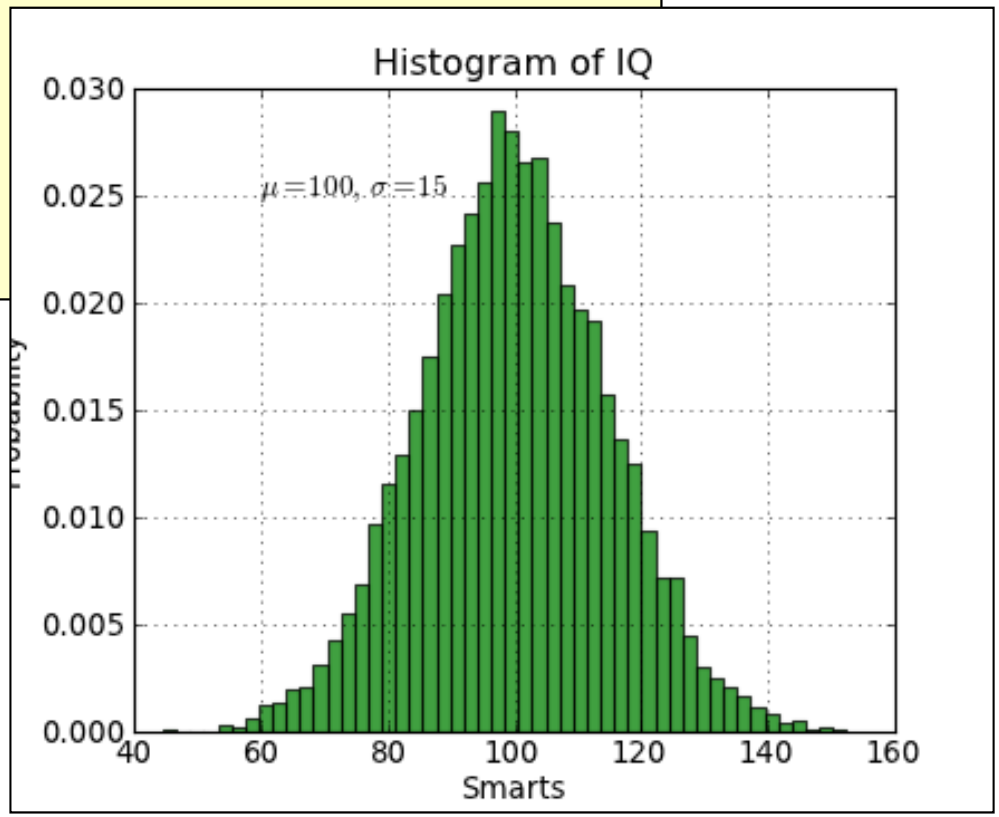
Matplotlib.pyplot example

```
import numpy as np
import matplotlib.pyplot as plt

mu, sigma = 100, 15
x = mu + sigma * np.random.randn(10000)

# the histogram of the data
n, bins, patches = plt.hist(x, 50, normed=1, facecolor='g', alpha=0.75)

plt.xlabel('Smarts')
plt.ylabel('Probability')
plt.title('Histogram of IQ')
plt.text(60, .025, r'$\mu=100,\ \sigma=15$')
plt.axis([40, 160, 0, 0.03])
plt.grid(True)
```



Working with text.
The text() command can be used to add text in an arbitrary location, and the xlabel(), ylabel() and title() are used to add text in the indicated locations.

Matplotlib.pyplot example

Bar chart.

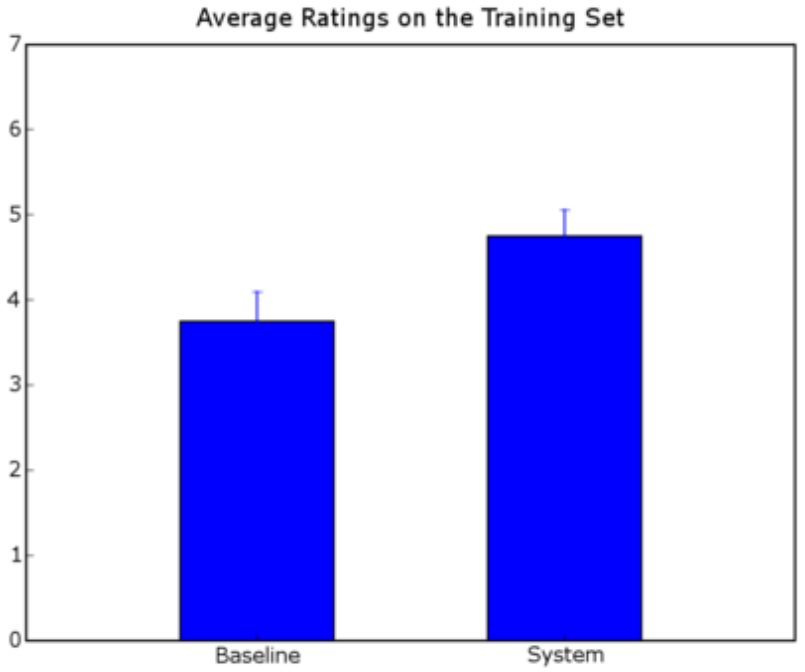
```
import numpy.numarray as na

from pylab import *

labels = ["Baseline", "System"]
data =   [3.75               , 4.75]
error =  [0.3497             , 0.3108]

xlocations = na.array(range(len(data)))+0.5
width = 0.5
bar(xlocations, data, yerr=error, width=width)
yticks(range(0, 8))
xticks(xlocations+ width/2, labels)
xlim(0, xlocations[-1]+width*2)
title("Average Ratings on the Training Set")

show()
```

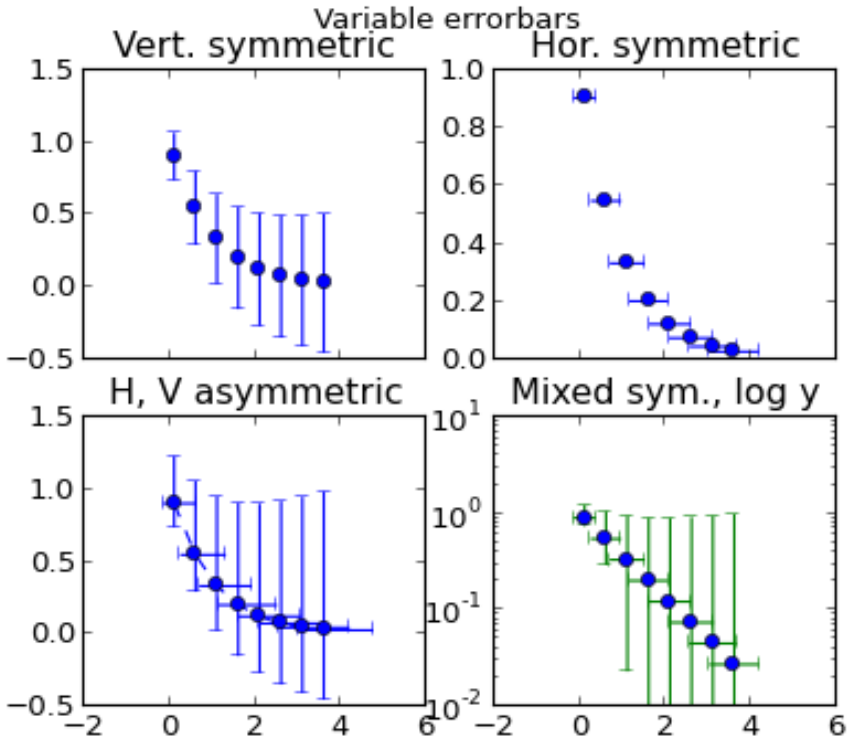


Matplotlib.pyplot example

Error bars.

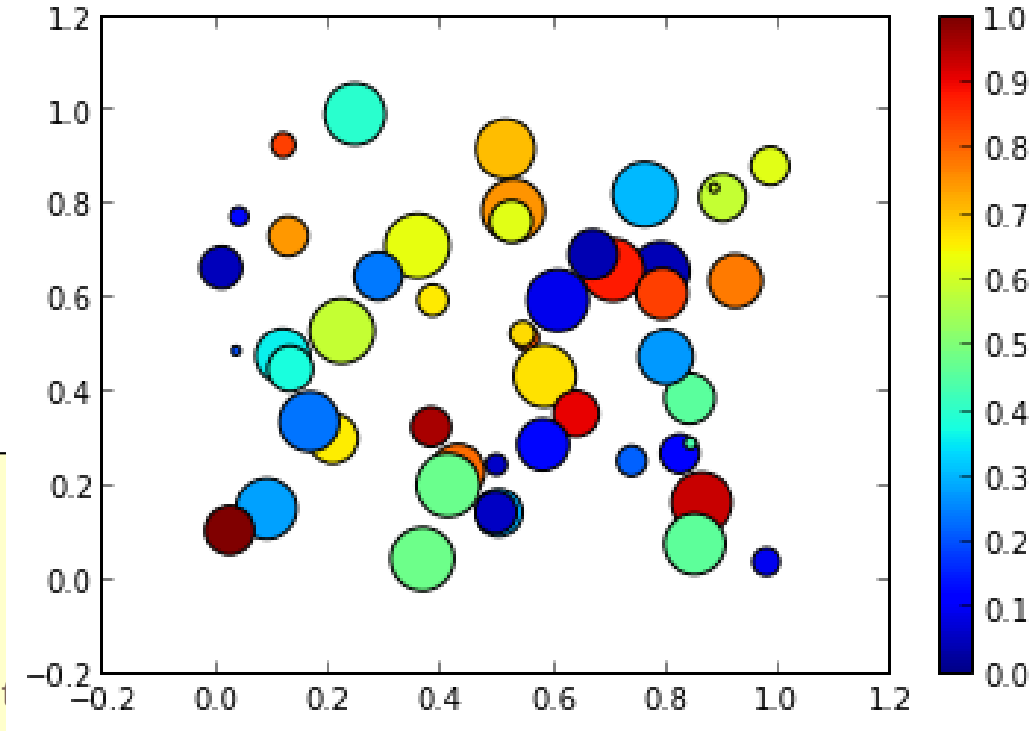
```
import numpy as np
import matplotlib.pyplot as plt

fig, axs = plt.subplots(nrows=2, ncols=2, sharex=Tr
ax = axs[0,0]
ax.errorbar(x, y, yerr=yerr, fmt='o')
ax.set_title('Vert. symmetric')
# With 4 subplots, reduce the number
# of axis ticks to avoid crowding.
ax.locator_params(nbins=4)
ax = axs[0,1]
ax.errorbar(x, y, xerr=xerr, fmt='o')
ax.set_title('Hor. symmetric')
ax = axs[1,0]
ax.errorbar(x, y, yerr=[yerr, 2*yerr], xerr=[xerr, 2*xerr], fmt='--o')
ax.set_title('H, V asymmetric')
ax = axs[1,1]
ax.set_yscale('log')
# Here we have to be careful to keep all y values positive:
ylower = np.maximum(1e-2, y - yerr)
yerr_lower = y - ylower
ax.errorbar(x, y, yerr=[yerr_lower, 2*yerr], xerr=xerr,
                            fmt='o', ecolor='g')
ax.set_title('Mixed sym., log y')
fig.suptitle('Variable errorbars')
plt.show()
```



Matplotlib.pyplot examples

- Scatter plots




```
import numpy as np
import pylab as plt

x = np.random.random(50)
y = np.random.random(50)
c = np.random.random(50)  # color of point
s = 500 * np.random.random(50)  # size of points

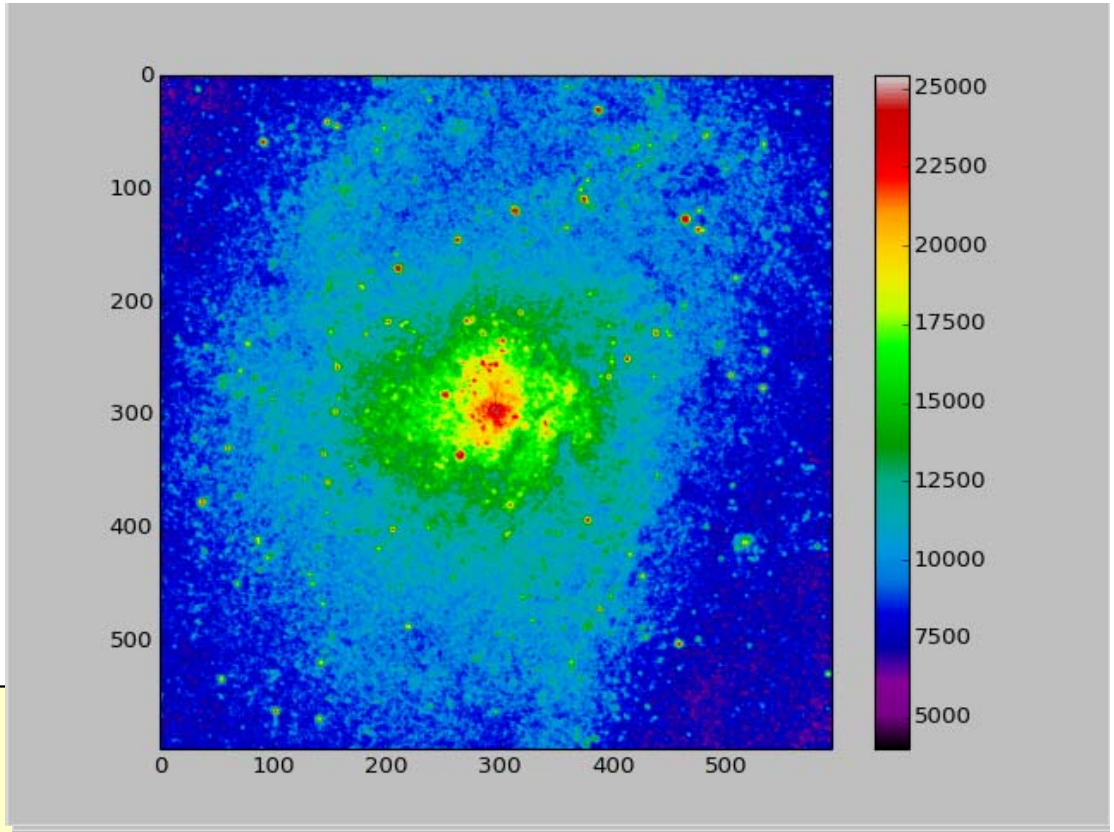
fig, ax = plt.subplots()
im = ax.scatter(x, y, c=c, s=s, cmap=plt.cm.jet)

# Add a colorbar
fig.colorbar(im, ax=ax)

# set the color limits - not necessary here, but good to know how.
im.set_clim(0.0, 1.0)
```

Matplotlib.pyplot basic example

Colormaps and contour figures are useful for plotting functions of two variables. In most of these functions we will use a colormap to encode one dimension of the data. There are a number of predefined colormaps.




```
>>> import pyfits
>>> import matplotlib.pyplot as plt
>>> data = pyfits.getdata('m33.fits')
>>> plt.imshow(data, cmap='gray')
<matplotlib.image.AxesImage object at 0x7f6c09b1a250>
>>> plt.colorbar()
<matplotlib.colorbar.Colorbar instance at 0x7f6c09aabb90>
>>> plt.show()
```

Matplotlib.pyplot example

3D plots.

```
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt

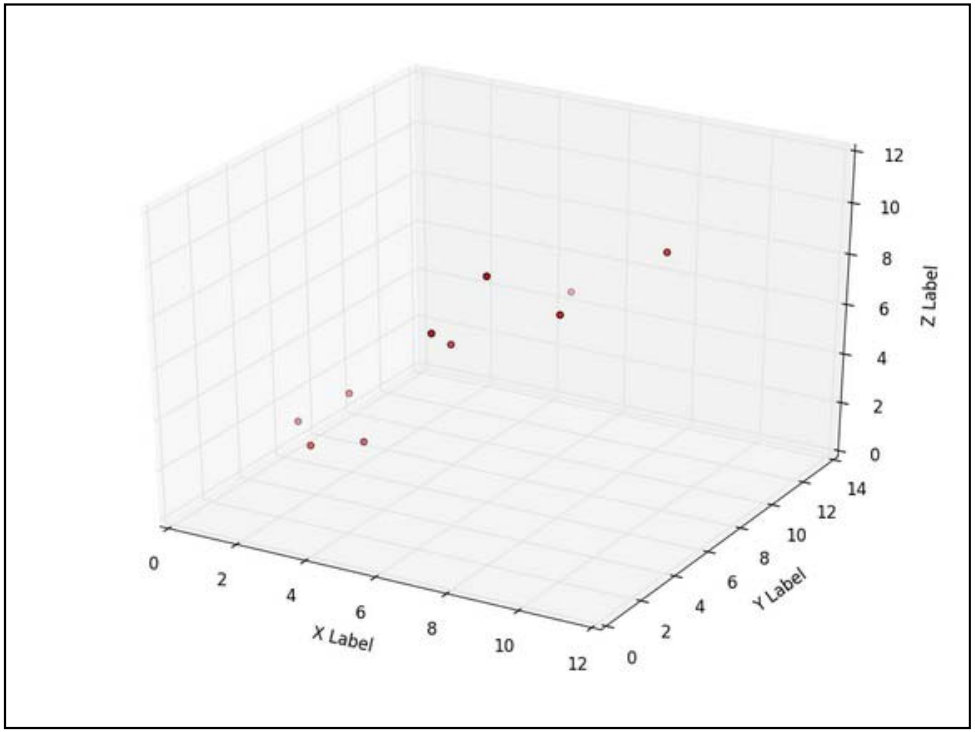
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

x =[1,2,3,4,5,6,7,8,9,10]
y =[5,6,2,3,13,4,1,2,4,8]
z =[2,3,3,3,5,7,9,11,9,10]

ax.scatter(x, y, z, c='r', marker='o')

ax.set_xlabel('X Label')
ax.set_ylabel('Y Label')
ax.set_zlabel('Z Label')

plt.show()
```



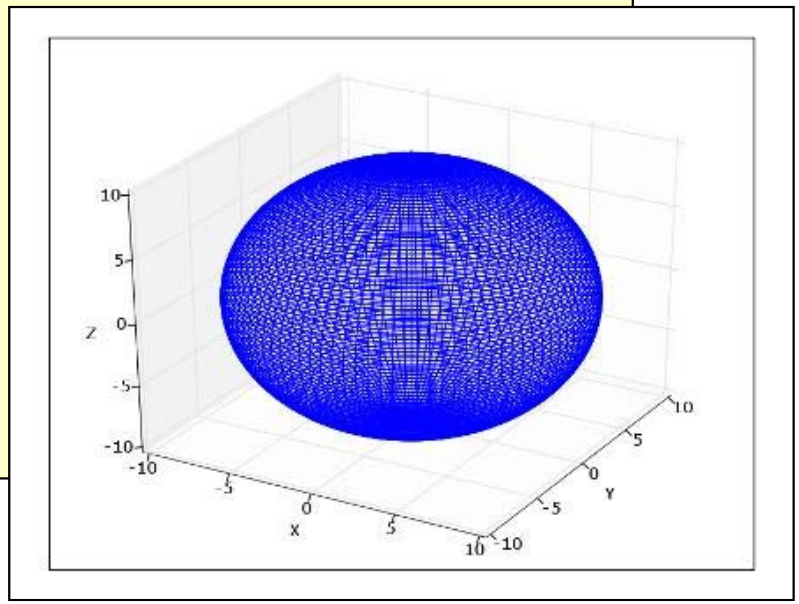
Matplotlib.pyplot example

3D plots.

```
from numpy import *
import pylab as p
import mpl_toolkits.mplot3d.axes3d as p3

# u and v are parametric variables.
# u is an array from 0 to 2*pi, with 100 elements
u=r_[0:2*pi:100j]
# v is an array from 0 to 2*pi, with 100 elements
v=r_[0:pi:100j]
# x, y, and z are the coordinates of the points for plotting
# each is arranged in a 100x100 array
x=10*outer(cos(u),sin(v))
y=10*outer(sin(u),sin(v))
z=10*outer(ones(size(u)),cos(v)

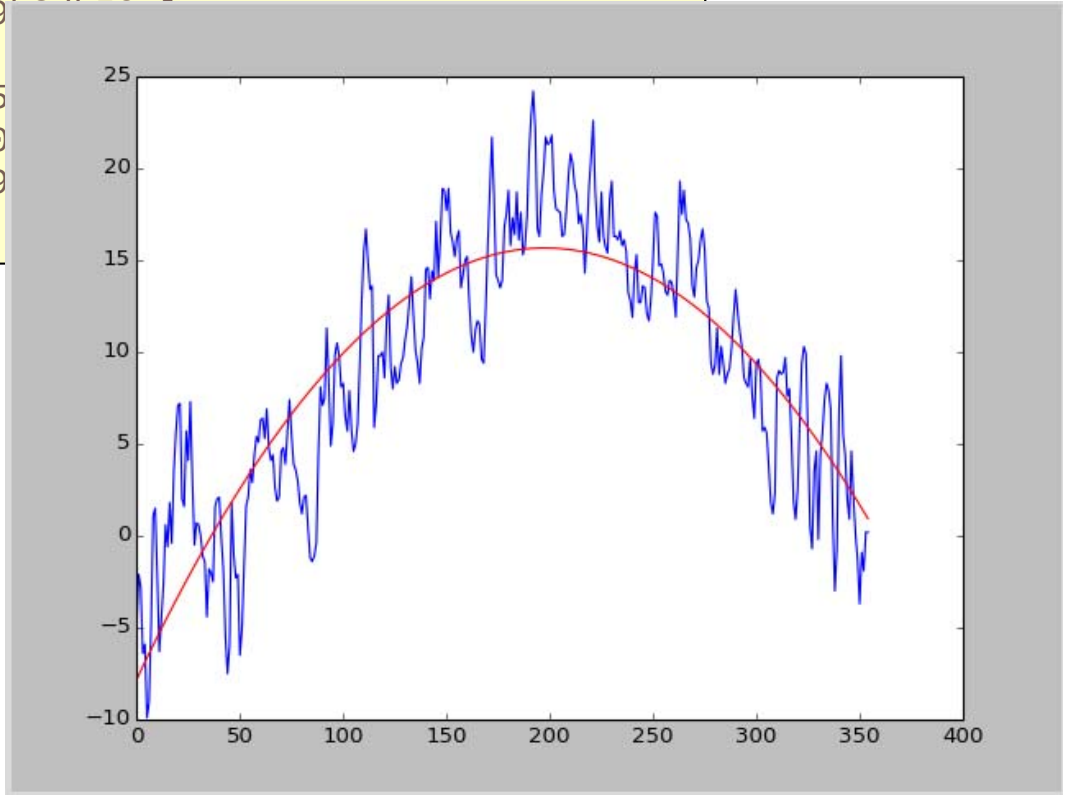
fig=p.figure()
ax = p3.Axes3D(fig)
ax.plot_wireframe(x,y,z)
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
p.show()
```



Data processing example

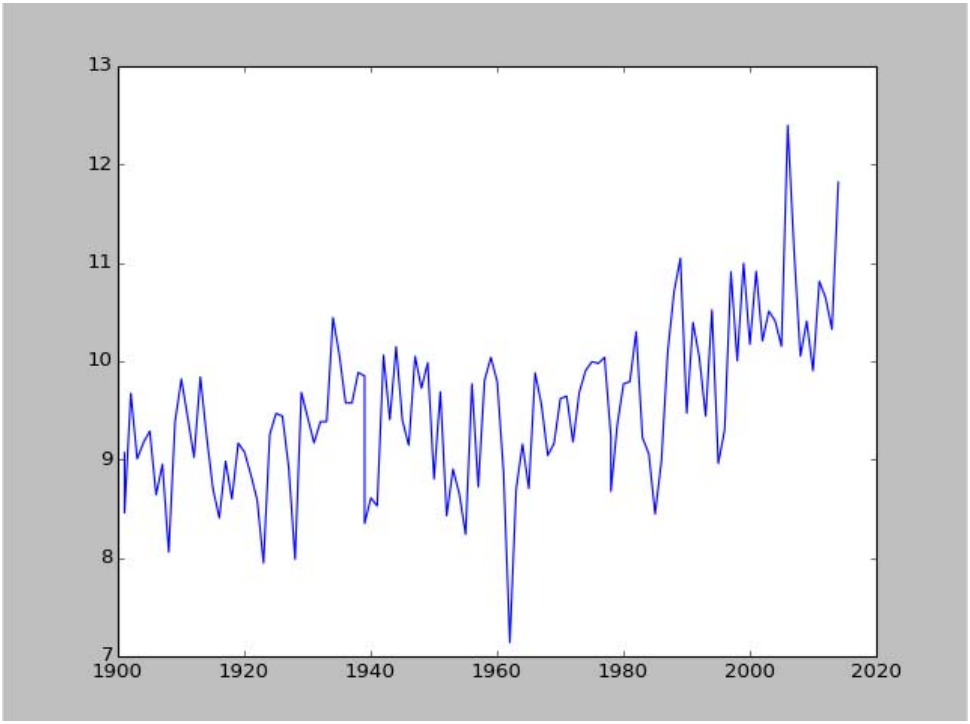
```
>>> bilt = np.genfromtxt('DeBilt.txt', delimiter=',',skip_header=1)
>>> temp=bilt[:,2]
>>> t=temp[0:355]

>>> coef=np.polyfit(range(355),t, 2)
>>> polyn=np.poly1d(coef)
>>> fit=polyn(range(355))
>>> plt.plot(range(355),t*0.1,'b',range(355),fit,'r')
[<matplotlib.lines.Line2D object at 0x7f6c09b2dd90>,
<matplotlib.lines.Line2D object at 0x7f6c09
>>> plt.show()
>>> plt.plot(range(355),t*0.1,'b',range(355
[<matplotlib.lines.Line2D object at 0x7f6c0
<matplotlib.lines.Line2D object at 0x7f6c09
>>> plt.show()
```



Data processing example

```
>>> m=[temp[i*355:(i+1)*355].mean() for i in range(temp.size / 356)]
>>> d=[str(datum[i*355])[0:4] for i in range(temp.size / 356)]
>>> plt.plot(d,m,'b')
[<matplotlib.lines.Line2D object at 0x7f6c10861b50>]
>>> plt.show()
```



Data processing example

```
>>> import numpy
>>> mm=numpy.array(m)
>>> mm.size
117
>>> coef=np.polyfit(range(117),mm, 3)
>>> polyn=np.poly1d(coef)
>>> mmfit=polyn(range(117))
>>> plt.plot(dd,mm,'b',dd,mmfit,'r')
[<matplotlib.lines.Line2D object at 0x7f6c09b2da90>,
<matplotlib.lines.Line2D object at 0x7f6c0d6
>>> plt.show()
```

```
>>> mmfit[-1]-mmfit[0]
2.1696593547888057
```

